INDEX

NAME: Prince Rohith A STD.: CSE SEC.: ROLL NO.: SUB.:

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1	28/7/26	Study of various network commands used in linux and windows	9	28/8/26
2	3/8/26	Study of Network cables	9	3/8
3	4/8/26	Experiments on cisco Packet Tracer	9	4/8
4	10/8/26	setup and configure a LAN using a switch and Ethernet cables in the lab	9	10/8
5	18/8/26	Experiments on packet capture Tool: Wireshark	9	18/8
6	7/9/26	Error correction at Data Link layer Hamming Code	9	7/9
7	15/9/26	Flow control at Data Link Layer Sliding Window Protocol	9	15/9

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
8	22/9/26	Nmap to Discover Live hosts using Nmap Scans	9	24/9
9	28/9/26	Implementation of subnetting in Cisco Packet Tracer	9	28/9
10		Internetworking with Routers in Cisco Packet Tracer	9	22/9
11		Routing at Network Layer	9	29/9
12	29/9/26	End to End communication at Transport Layer: Socket programming	(9)	29/9
13	30/9/26	Implement your own ping program	(9)	30/9
14	1/10/26	Write a code using Raw sockets to implement packet sniffing	(9)	1/10
15		Analyse Various Types of severs using Webalizes tools		

Ex No: 1
Date: 28/7/26

Study of various Network commands used in Linux and Windows.

AIM:

To study the various network commands used in Linux and windows.

COMMANDS:

1. arp -a : Address resolution protocol, It show IP address along with IP and MAC address of router.

2. hostname : It displays the name of the computer.

3. ipconfig : It displays detailed config information about TCP/IP connection

4. nbtstat -a : solves problem with NETBIOS name resolution.

5. netstat : displays variety of network stats.

6. nslookup : It is a tool for DNS lookup. returns IP address and MX, NS records

7. pathping: it traces the route the packet send to.

8. Ping: Testes connectivity between two nodes.

9. Route: used to show/manipulate IP routing table.

Linux commands:

1. ip: basic command used to show all the interfaces. Adds IP to interface.

2. ifconfig: used for troubleshooting the network.

3. mtr: It traces the route the packet send to.

4. tcpdump: helps to monitor the tcp packets send and received

5. ping: same as Windows.

6. nmcli: If you are connected over ethernet, you can use nmcli to configure.

Examples

1) arp -a

Interface: 192.168.1.10 -0x6

Internet address	Physical address	Type
192.168.1.1	00-14-22-01-23-45	dynamic

2) hostname

DESKTOP-ABC123

3) ipconfig /all

Ethernet adapter Local Area connection:

IPV4 Address . . . . . : 192.168.1.10

Subnet Mask . . . . . : 255.255.255.0

4) NetBIOS status

nbtstat -a DESKTOP-ABC123

NetBIOS Remote Machine NameTable

Name	Type	status
DESKTOP-ABC	<00> unique	Registered

5) Network

net-stat -a

Proto Local Address Foreign Address

TCP 192.168.1.10:5000 142.250.183.78

6) DNS lookup

nslookup www.google.com

Server: 8.8.8.8

Name: www.google.com

7) Ping

ping google.com

64 bytes from 142.250.183.78

8) ip a

Shows the IP, mac address of interfaces

9) ip route

default via 192.168.1.1 dev eth0

192.168.1.0/24 dev eth0 proto kernal scope link src 192.168.1.105

10) hostname -I

192.168.1.105

11) ss -tuln

Netid	State	Local address:Port
tcp	LISTEN	0.0.0.0:22
tcp	LISTEN	0.0.0.0:80
udp	UNCONN	0.0.0.0:68

12) curl ifconfig.me

49.205.34.189

13) dig google.com

;; Question Section:

;google.com. IN

;; Answer Section:

google.com. 300 IN A 142.250.183.14

RESULT:

Thus the basic commands for Linux and windows is studied and executed output is achieved.

28/2/24

Ex No: 2 Study of Network Cables
Date: 3/7/26

AIM:

Study of different types of network cables:

a) Understanding different types of network cable.

1. Unshielded Twisted Pair (UTP) cable.
2. Shielded Twisted Pair (STP) cable
3. coaxial cable.
4. Fibre optic cable.

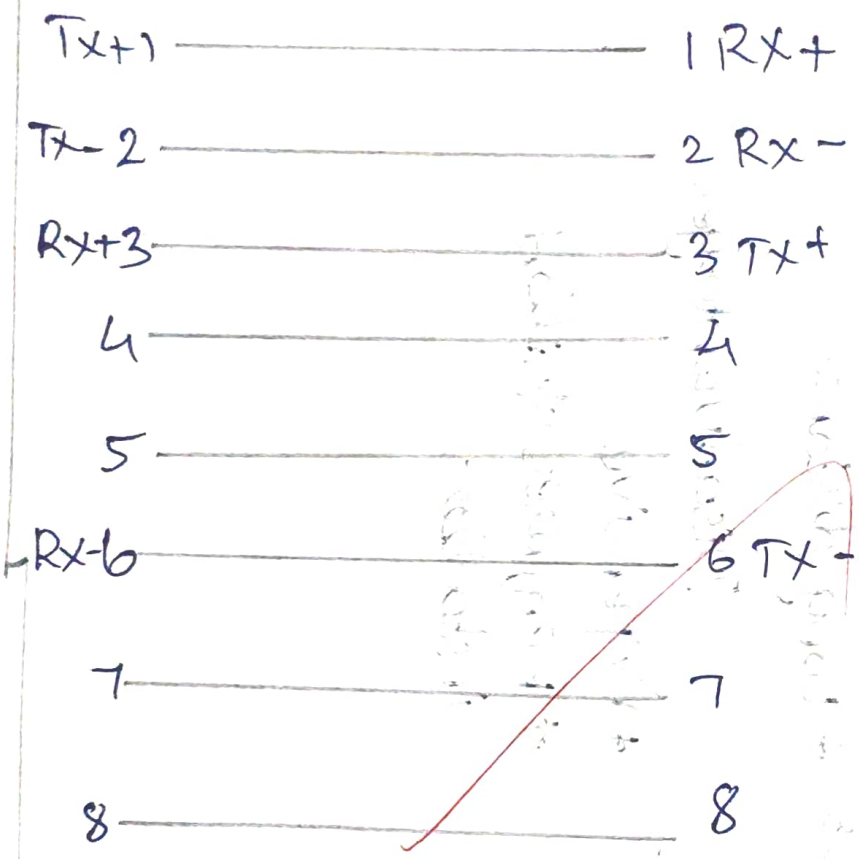
Cable Type	Category	Maximum Data Transmission	Advantages / Disadvantages	Application / use
UTP	Category 3	10 Mbps	**Advantages** * Cheaper in cost * Easy to install as they have a smaller overall diameter **Disadvantages** * More prone to (EMI) Electromagnetic interference and noise.	10 Base -T Ethernet
	Category 5	up to 100 Mbps		Fast Ethernet, Gigabit Ethernet
	Category 5e	1 Gbps		Fast Ethernet, Gigabit Ethernet
STP	Category 6, 6a	10 Gbps	**Advantages** * Shielded * Faster than UTP * Less susceptible to noise and interference	Gigabit Ethernet, 10G Ethernet (55m) Widely used in data centres.

SSTP	category 7	10Gbps	Disadvantages * Expensive * Greater installation effort	Gigabit Ethernet, 10G Ethernet (100m)
Coaxial cable	RG-6 RG-59 RG-11	10-100Mbps	Advantages * High bandwidth * Immune to interfer-ence * Low loss bandwidth * Versatile Disadvantages * Limited distance * cost * size is bulky	Speed of signal is 500m Television network High speed internet connections

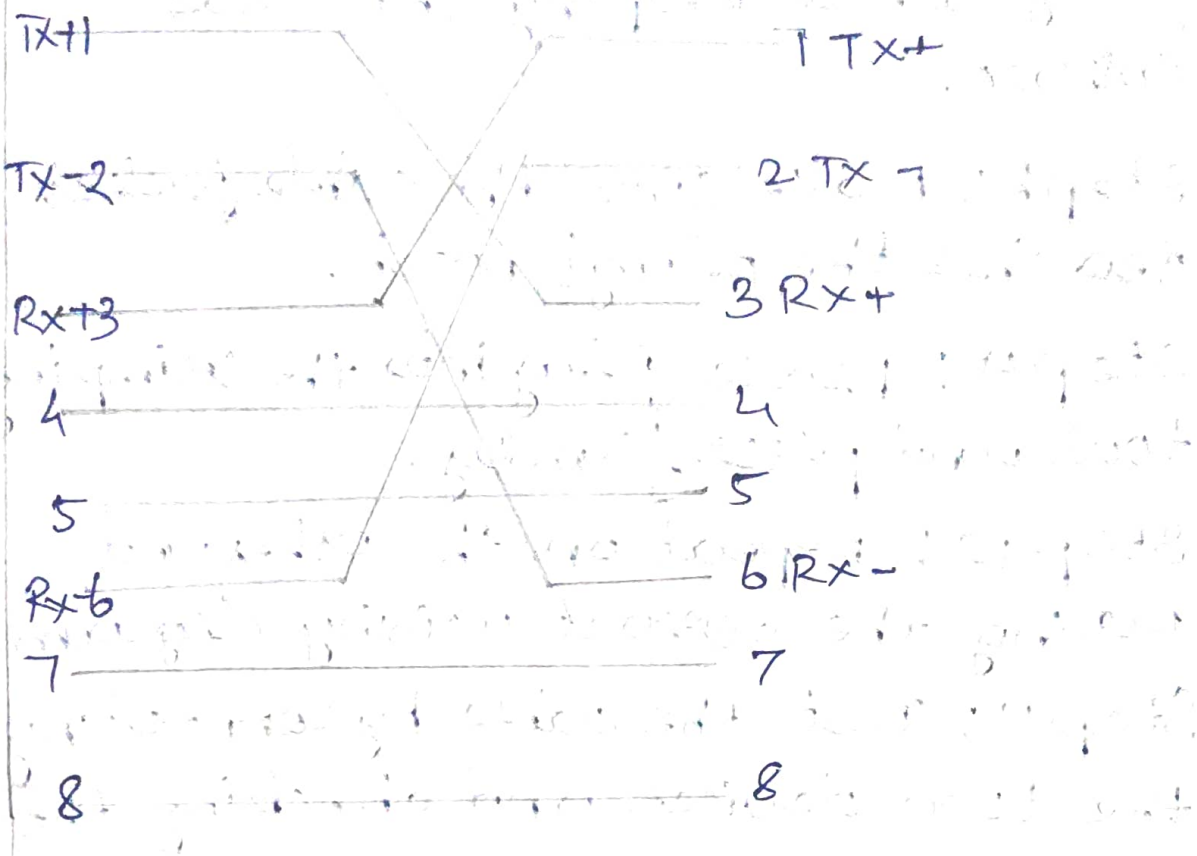
Fibre optics cable	Simple mode Multimode	100Gbps	Advantages * High speed * High bandwidth * High security * Long distance Disadvantages * Expensive * Requires skilled installers	Maximum distance of fibre optics cable is around 100 meters.

Make your own Ethernet cross over cable / straight cable:

Straight thru cable



X-over cable



Step 1: Put the shields into the cable

Step 2: Strip about 1.5cm of the outer cable covering from both ends.

Step 3: Seperate the 4 twisted pairs of wires.

Step 4: Arrange the wires according to the required diagram:

A → A = straight-through cable

A → B = crossover cable

Step 5: straighten, the line up the wires in the correct order.

Step 6: Trim the wire ends so they are all the same length.

Step 7: Push the wires into the RJ45 plug while keeping their order correct.

Step 8: Make sure the cable jacket reaches the plug's notch.

Step 9: Place plug into the crimping tool and press firmly.

Step 10: Repeat on the other end using the correct wiring diagram.

Step 11: Test the cable by connecting the two devices and checking the connection.

RESULT:

Thus the different types of network cables and how to connect them is studied.

Ex No: 3. CISCO PACKET TRACKER

Date: 4/8/26

AIM:

To study the cisco packet tracer simulation tool and analyse the behaviour of network devices such as hubs and switchs by defining a simple network and observing packet transmission.

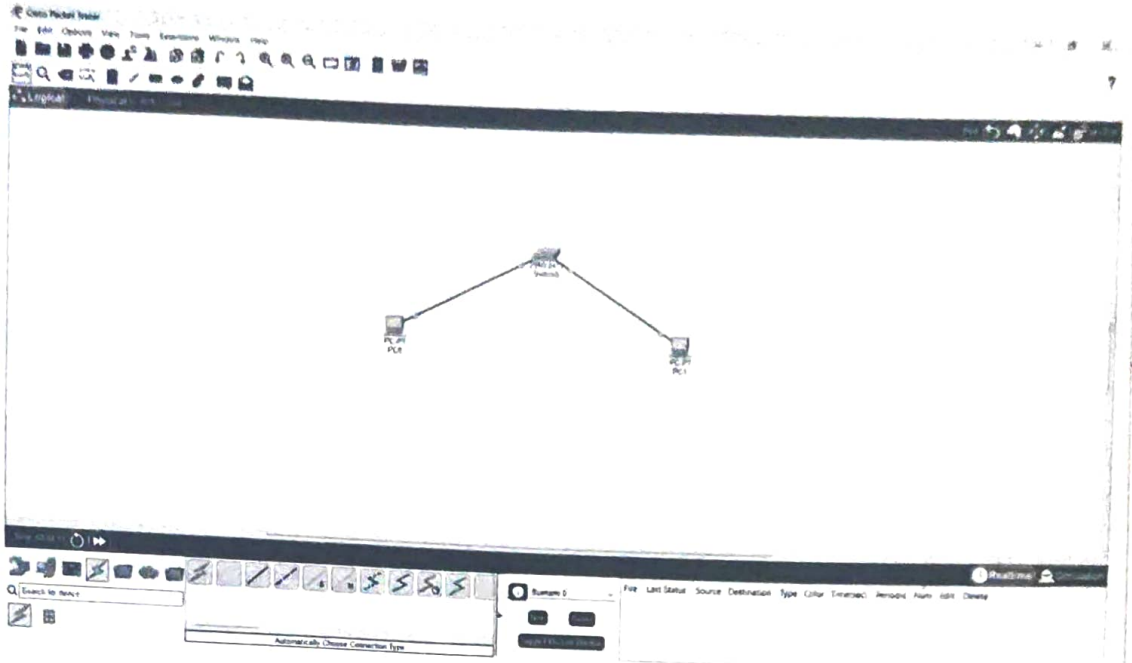
PROCEDURE:

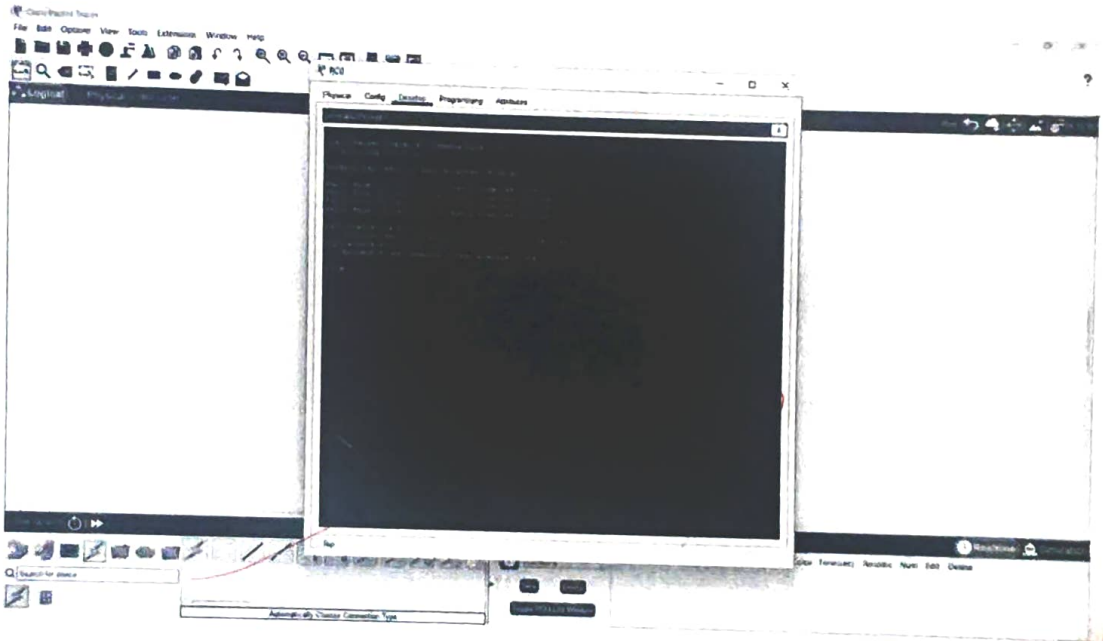
Part A: Study of packet tracer

1. Open the cisco packet tracer application.
2. Observe the main interface, workspace, toolbar and network device panal.
3. Select network devices such as PCs, hubs and switches from the device panel.
4. Place the selected devices in the logical workspace.
5. Select the appropriate Ethernet cable and connect the devices.
6. Observe the connection statuss and use simulation mode to analyse network communication.

Part B: Network using a Hub

1. Open a new project in Cisco Packet Tracer.
2. Place four generic PCs and one hub in the workspace.
3. Select copper straight-Through cable from the connection menu.
4. Connect each PC to the hub using the cable.
5. Open each PC, select the Desktop tab and choose IP configuration.
6. Assign IP addresses and subnet masks to all four PCs
7. Select the Add simple PDU tool
8. Click the source PC and then the destination PC to send a test packet
9. Observe the packet transmission through the hub and note how the hub forwards the packet to its connected ports.





Part C: Network using a switch

1. Create another network with four generic PCs and one switch.
2. connet all PCs to the switch using copper straight Through cables.
3. Assign IP addresses and subnet masks to the PCs in the same subnet.
4. Use the add simple PDU tool to send a packet between two PCs.
5. Observe the packet transmission in simulation mode.
6. Note how the switch forwards the packet towards the destination using its MAC address table.

RESULT:

Thus the cisco Packet tracer was studied and used to design and simulate hubbased and switch based networks.

Ex No: 4
Date: 10/8/26

LAN configuration

AIM:

To set up and configure a LAN using a switch and Ethernet cables.

PROCEDURE:

1. Connect four computers to a network switch using ethernet cables.
2. Assign IP addresses and subnet masks to each computer as follows:

   10.1.1.1, 10.1.1.2, 10.1.1.3 and 10.1.1.4 as IP Address and 255.0.0.0 as subnet mask for all the PCs.

3. Ensure all computers are connected properly to the switch.
4. Use the ping command to check connectivity between the computers.
5. Share a folder on one computer and access it from another computer on the same LAN.

OUTPUT:

* All computers are connected through the switch
* successfully ping replies confirm network connectivity.
+ Shared folders can be accessed by other computers when sharing permissions are configured.

RESULT:

A LAN was configured using a switch and ethernet cables, and connectivity between the computers was tested successfully.

10

Ex No: 5
Date: 18/8/26

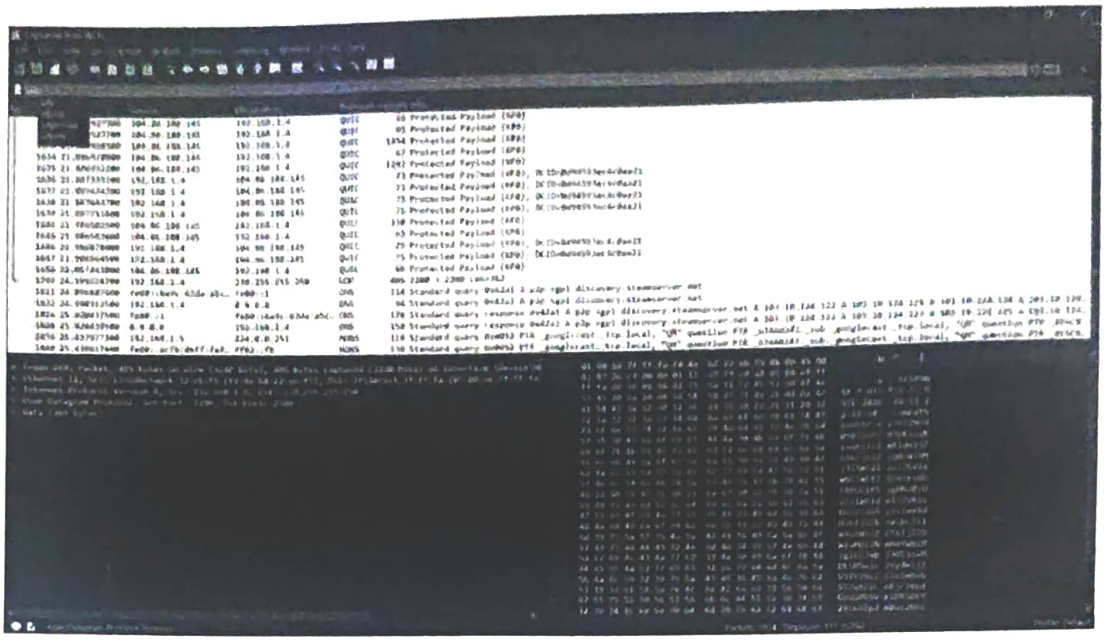
Wireshark

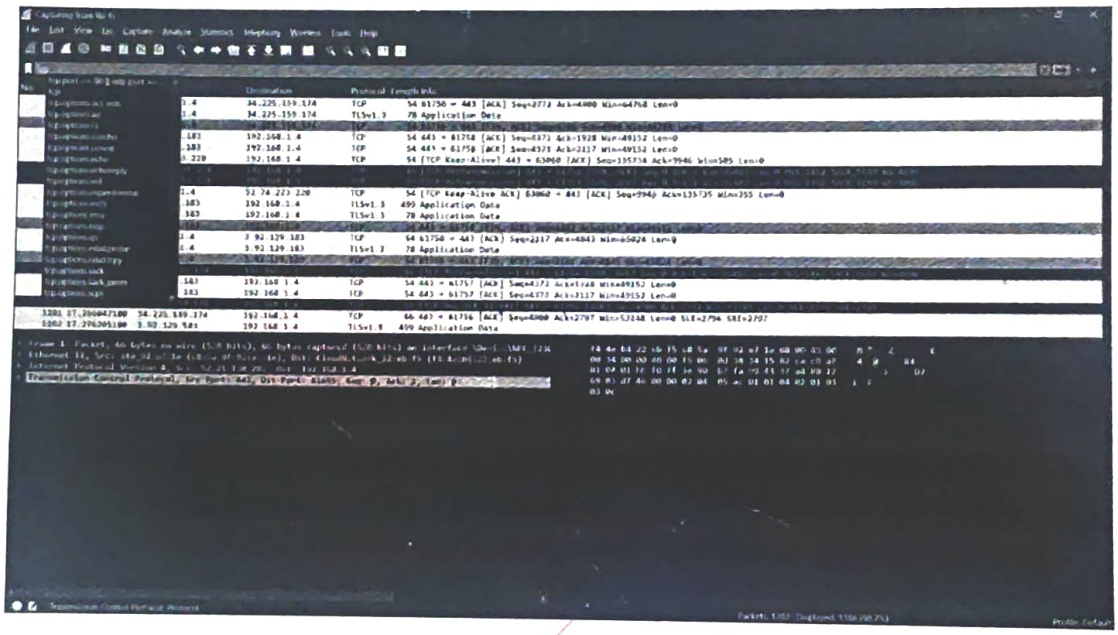
AIM:

To capture and analyse network packets using wireshark and study the encapsulation of data at different layers of the protocol stack.

PROCEDURE:

1. Open wireshark and select the active network interface.
2. Start packet capture and generate network traffic by browsing websites or using network commands.
3. Observe the captured packets in the packet list.
4. Select a packet to view its protocol details and raw bytes.
5. Apply display filters such as dns, tcp or icmp to view specific packet.
6. Stop the capture and observe the communication between protocol layers.





RESULT:

Network packets were captured and analysed using wireshark and the protocol details at different layers were studied.

Exp No: 6
Date: 7/9/24

Error Detection and Correction using Hamming co

AIM:

To implement error detection and correction using the hamming code technigue.

PROCEDURE

1. Enter a text message and convert it into binary data.
2. Encode the data using Hamming code and transmit it through a simulated channels.
3. At the receiver, detect and corre single-bit errors and recover the original message.

CODE:

```
def encode(data):
    bits = [0] + data
    bits[3] = bits[1] ^ bits[2] ^
                        bits[4]
    bits[5] = bits[1] ^ bits[2] ^
                        bits[6]
```

```
    bits[7] = bits[1] ^ bits[4] ^ bits[6]
    return bits[1:8]
def decode(code):
    b = [0] + code
    p1 = b[1] ^ b[3] ^ b[5] ^ b[7]
    p2 = b[2] ^ b[3] ^ b[6] ^ b[7]
    p4 = b[4] ^ b[5] ^ b[6] ^ b[7]
    error = p1 + 2 * p2 + 4 * p4
    if error:
        b[error] ^= 1
        print("Error corrected at
              bit:", error)
    return [b[3], b[5], b[6], b[7]]
message = input("Enter a message:")
data = ''.join(format(ord(c), '08b')
    for c in message)
encoded = []
for i in range(0, len(data), 4):
    encoded.extend(encode([int(x)
 for x in data[i:i+4]]))
print("Encoded data:", ''.join(map(
 str, encoded)))
```

#Simulation

```
pos = int(input("Enter bit position
to corrupt (0 for none):"))
If 1<= pos <= length(encoded):
    encoded[pos-1]^=1
decoded=[]
for i in range(0, len(encoded),7):
    decoded.extend(decode(encoded
                          [i:i+7]))
binary = ''.join(map(str, decoded))
recovered = ''.join(...
    chr(int(binary[i:i+8],2))
    for i in range(0, len(binary),8)
)
print("Received Message:",
                    recovered)
```

OUTPUT:

Enter a message: A
Encoded data: 1001100101
Enter bit position to corrupt (0 for none): 3

Error corrected at bit: 3
Received message: A

RESULT:

Hamming code was implemented successfully to detect and correct single-bit errors and recover the original message.

6/9/25

Exp No: 7 Sliding Window Protocol
Date: 15/9/24

AIM:
To implement flow control at the data link layer using the sliding window protocol.

CODE:

```
def sliding_window():
    message = input("Enter Message:")
    window_size = int(input("Enter
        window size: "))
    frames = list(enumerate(message,
        start=1))
    total = len(frames)
    base = 0
    while base < total:
        end = min(base + window_size,
                  total)
        print("\nSending frames:")
        for i in range(base, end):
            print(f"Fame {frames[i][0]
{frames[i][1]}")
```

```
        ack = input("Enter ACK number
(${frames[end-1][0]}) or NACK: "
        ).strip().upper()
        if ack == "NACK":
            print("Frames will be re
            transmitted.")
        else:
            try:
                ack_number = int(ack)
                if frames[base][0] <=
ack_number <= frames[end-1][0]:
                    base = ack_number
                    print("Acknowlegement
received.")
                else:
                    print("Invalid ACK.
Retransmitting frames.")
            except ValueError:
                print("Invalid Input.
Retransmitting frames.")
    print("\nAll frames transmitted
successfully.")
sliding_window()
```

OUTPUT:

Enter message: HELLO
Enter window size: 3

Sending frames:
frame 1: H
frame 2: E
frame 3: L
Enter ACK number (3) or NACK: 3
Acknowledgement received.

Sending frames:
frame 4: L
frame 5: O
Enter ACK number (5) or NACK: 5
Acknowledgement received.
All frames transmitted successfully

RESULT:

The sliding window protocol was implemented to demonstrate frame transmission, flow control and acknowledgement handling.

Ex No: 8
Date: 22/9/24

Discover Live Hosts Using Nmap

AIM:

To discover live hosts using Nmap scans (ARP, ICMP, TCP/UDP) on the TryHackme platform.

PROCEDURE:

1. open the Nmap room in TryHackMe and launch the AttackBox. Identify the target IP address and understand the different host-discovery techniques used by Nmap.
2. Perform host discovery using the following commands on authorized lab targets.

ARP Scan

nmap -PR -sn <TARGET>

ARP requests identify active hosts on the same local subnet.

ICMP Scan

nmap -PR -sn <TARGET>

ICMP Echo requests are sent to identify hosts that respond.

TCP SYN Ping

nmap -PS80,443 -sn <TARGET>

TCP SYN probes are sent to the specified ports to check whether the host is reachable.

UDP Ping

nmap -PU53 -sn <TARGET>

UDP probes are used to identify reachable host through their responses.

3. Observe the results of each scan. Compare the hosts discovered using the different techniques and notes that firewalls or network configurations may affect the results.

OUTPUT: 1. ARP Scan

nmap -PR -sn 10.10.10.5

Starting Nmap

Host is 10.10.5 appears to be up.

Nmap done: 1 IP address scanned

2. ICMP Scan

nmap -PE -Sn 10.10.10.5

Starting Nmap

Host 10.10.10.5 appears to be up.

Nmap done: 1 IP address scanned

3. TCP SYN Scan

nmap -PS80,443 -sn 10.10.10.5

starting Nmap

Host 10.10.10.5 appears to be up

NMap done: 1 IP address scanned.

RESULT:

Live-host discovery using ARP, ICMP, TCP SYN, and UDP scans was studied and performed using Nmap on the TryHackMe platform.

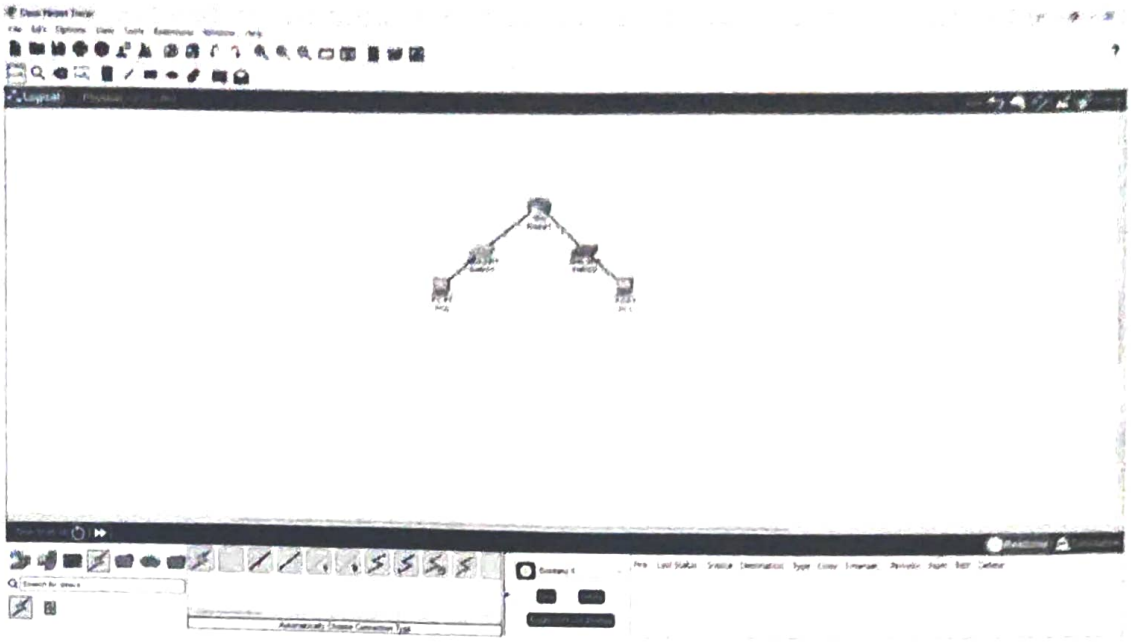
Exp No: 9 Implementation of
Date: 28/9/24 Subnetting in Cisco Packet
Tracer

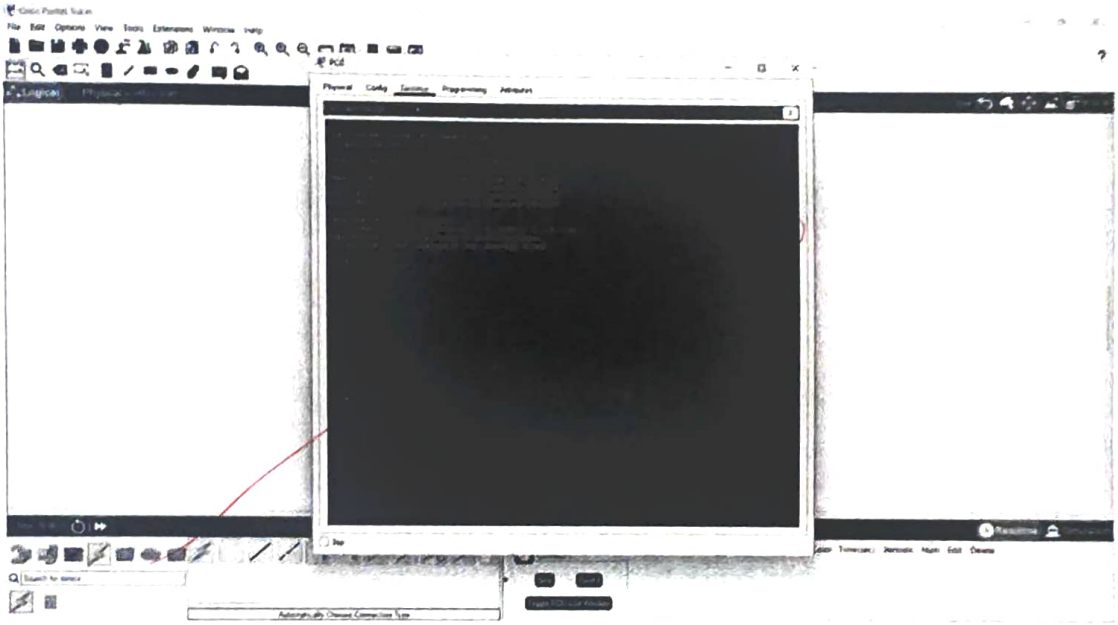
AIM:

To implement subnetting in cisco Packet Tracer, assign static IP addresses, and test communication between subnets.

PROCEDURE:

1. Create a network topology using PCs switches and a router in Cisco Packet Tracer. Connect the devices using Ethernet cables.
2. Divide the network 192.168.1.0/24 into /27 subnets with subnets mask 255.255.255.224. Assign IP addresses and default gateway to PCs and configure the router interfaces.





3. Test connectivity between PCs using Ping, first with the router connected and then without router connected.

Output

Ping 192.168.1.34

Reply from 192.168.1.34

Packets: Sent=4, Received=4, Lost =0.

RESULT:

Subnetting was implemented success
and communication between diff
Subnets was tested using Cisco
Packet Tracer

Exp No: 10 Internetworking Using
Date: Routers in Cisco Packet
Tracer

AIM:

To design and configure an internetwork using a router and test communication between different networks.

PROCEDURE:

1. Create a topology with one router and two PCs in Cisco packet Tracer. connect each PC to a different router interface using copper straight-through cables.

2. configure the router interfaces with PCs with the following static IP addresses.

192.168.10.2 (PC1), 192.168.10.1 (Router Fa0/0), 192.168.20.1 (Router Fa0/1)

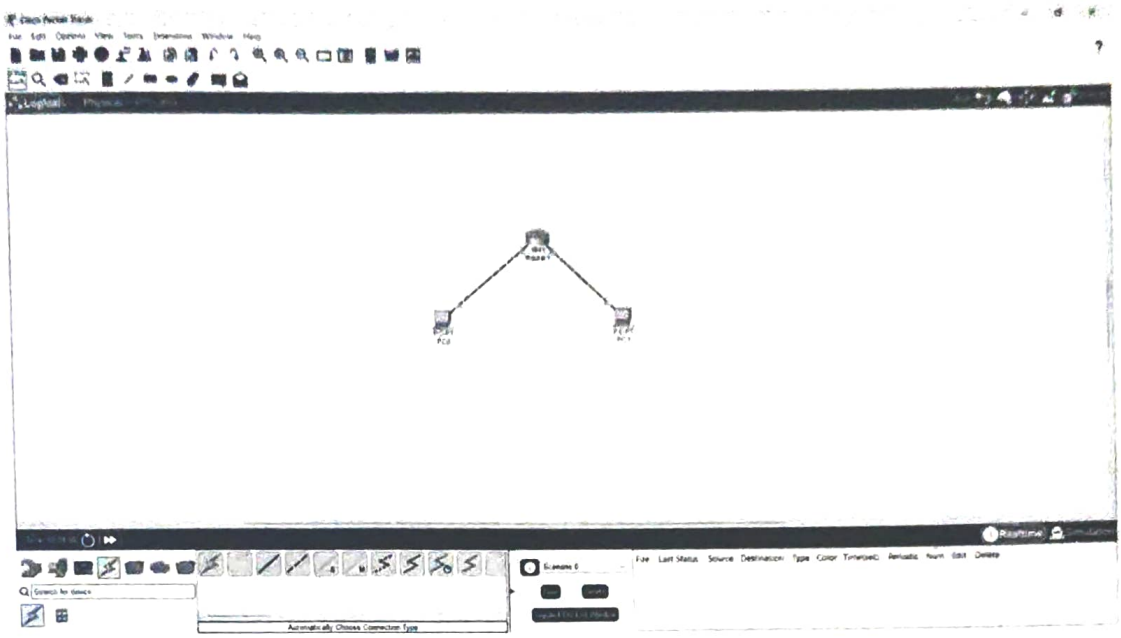
192.168.20.2 (PC2).

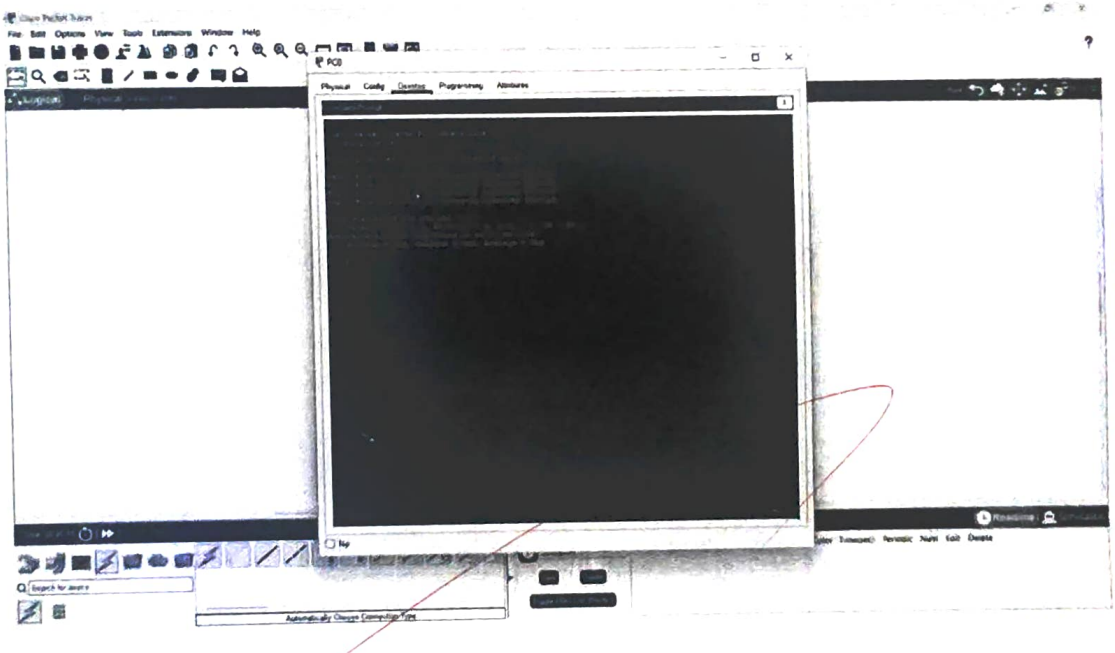
3. Use the simulation mode of the ping command to test commu between PC1 and PC2.

Output:

Ping 192.168.20.2

Reply from 192.168.20.2:
Bytes=32 Time<1ms TTL=127
Packets: Sent=4, Received=4, lo





RESULT:

An internetwork was configured using a router in Cisco packet tracer, and communication between two different networks was tested successfully.

Exp No: 11 Routing at Network Layer - Static Routing and RIP

Date:

AIM:

To simulate static routing and RIP (Routing information protocol) using Cisco packet Tracer.

PROCEDURE:

1. Creating a network topology using three routers and two PCs. Connect the routers using Serial links and assign IP addresses to all router interfaces and PCs according to the addressing table.

2. Static Routing: configure routing manually using the ip route command on each router. Verify the routing table using show ip route.

RIP Routing: Enable RIP on each router and advertise the directly connected networks.

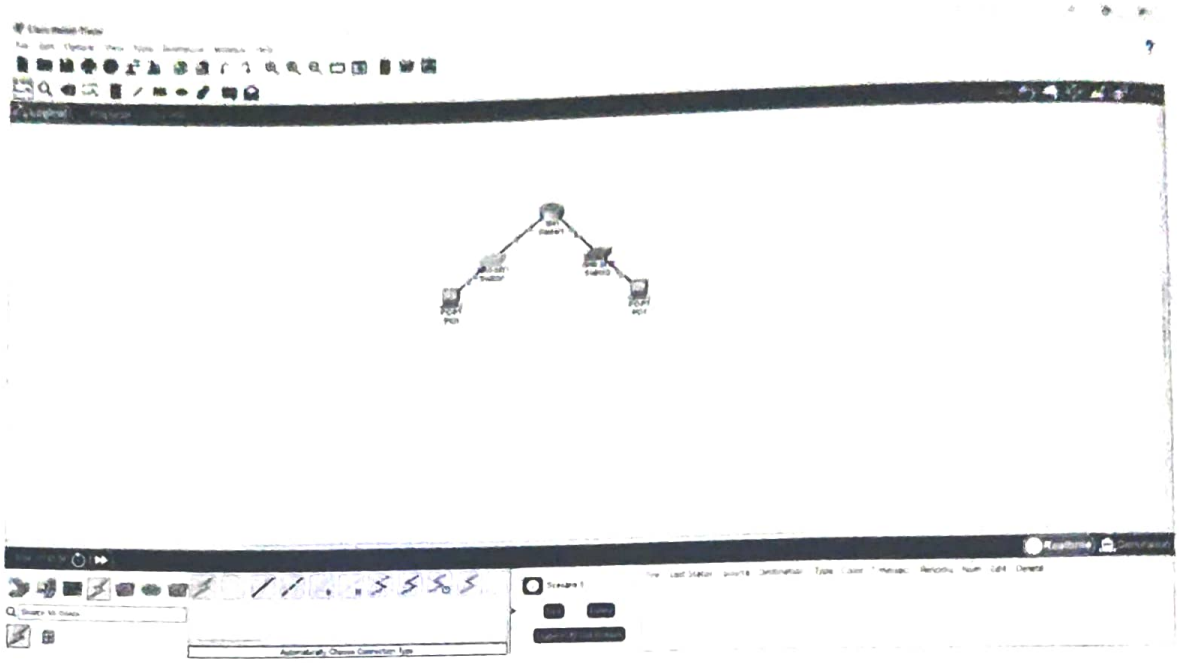
Router(config)# router rip
Router(config-router)# network
10.0.0.0
Router(config-router)# network
192.168.1.248

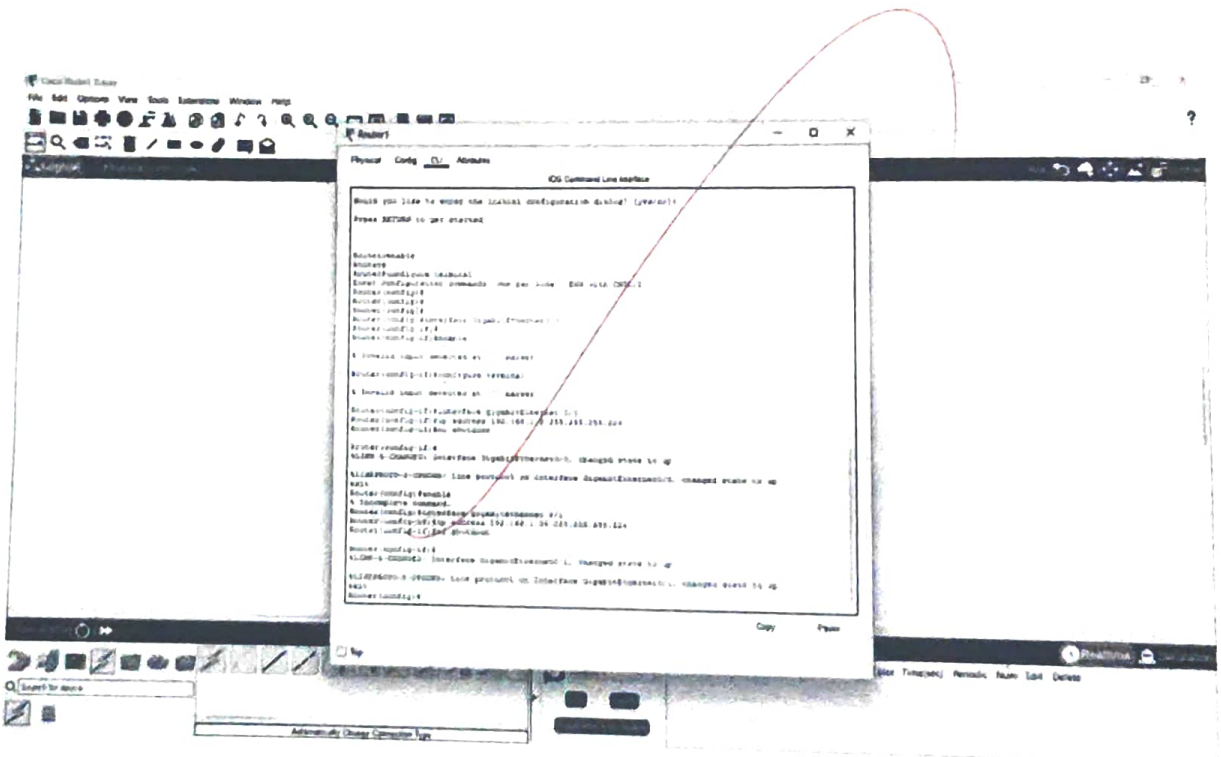
3. Test connectivity between the PCs using ping. Disconnect a route and observe how RIP selects an alternative route when one is available.

OUTPUT:

Ping 20.0.0.2

Reply from 20.0.0.2:
Bytes=32 Time<1ms TTL=125
Packets: Sent=4, Received=4,
lost=0





RESULT:

Static Routing and RIP were simulated in Cisco Packet Tracer and connectivity between different networks was tested successfully

Exp No: 12 Implement Echo and
Date: 29/9/24 Chat client-server using
TCP Socket.

AIM:

To implement an echo and a chat client serve program using TCP Socket

Echo:

code:

Server

```
import socket
S=socket.socket()
S.bind(("127.0.0.1", 5000))
s.listen(1)
print("Server waiting...")
conn, addr = s.accept()
Print("Connected:", addr)
while True:
    data = conn.recv(1024)
    if not data:
        break
    conn.sendall(data)
```

```
conn.close()
s.close()
```

Client

```
impost socket
s=socket.socket()
s.connect(("127.0.0.1", 5000))
msg=input("Enter message:")
s.sendall(msg.encode())
data=s.recv(1024)
print("Echo from server:" data.decode())
s.close()
```

output:

Enter Message: Hello Server
server waiting...
connected: ('127.0.0.1', 54321)
Echo from server: Hello Server

Chat

```
import socket
import time

host = input ("Enter IP address: ")

try:
    s = socket.socket(
        socket.AF_INET,
        socket.SOCK_DGRAM,
        socket.IPPROTO_ICMP
    )
    s.settimeout(2)

    start = time.time()
    s.sendto(b"Ping Test", (host, 1))
    data, addr = s.recvfrom(1024)
    end = time.time()

    print("Reply from:", addr[0])
    print("Responce time:",
        round((end-start)*1000, 2),
            "ms")
except OSError:
    print ("Host unreachable or
        no reply received.")
```

```
finally:
    if 's' in locals():
        s.close()
```

OUTPUT:

Enter IP address: 127.0.0.1
Reply From: 127.0.0.1
Response time: 0.15ms

RESULT:

Thus, the ping Program was implemented to test host reachability and measure responce time.

Exp No: 13 Implement Your own
Date: 30/9/26 Ping program

AIM:
To implement a ping program in python to check the reachability of a host using ICMP.

CODE:

```
import socket
import time

host = input("Enter IP address: ")

try:
    s = socket.socket(
        socket.AF_INET,
        socket.SOCK_DIAGRAM,
        socket.IPPROTO_ICMP
    )
    s.settimeout(2)

    start = time.time()
    s.sendto(b"Ping Test", (host, 1))
    data, addr = s.recvfrom(1024)
    end = time.time()
```

```
        print("Reply from:", addr[0])
        print("Responce time:",
              round((end-start)*1000,2),
                    "ms")
    except OSError:
        print("Host unreachable or
              no reply received.")
    finally:
        if 's' in locals():
            s.close()
```

OUTPUT:

```
Enter IP address: 127.0.0.1
Reply from: 127.0.0.1
Responce time: 0.15ms
```

RESULT:

Thus the ping program was implemented to check host reachabl and measure response time.

Exp No: 14 Packet Sniffing Using
Date: 1/10/26 Raw Sockets

AIM:

To write a program using raw sockets to capture and analyse network packets.

CODE:

```
import socket
import struct

S= socket.sockets(.
        socket.AF_INET,,
        socket.SOCK_RAW,
        socket.IPPROTO_IP
)
s.bind(("127.0.0.1",0))
S.setsockopt(socket.IPPROTO_IP,
Socket.IP_HDRINCL, 1)
Print("packet sniffer started...")
try:
    while True:
        packet, addr = s.recvfrom
                        (65535)
```

```
        ip_header = packet[:20]
        iph = struct.unpack("!BBHH
            HBBH4s4s", ip_header)

        source = socket.inet_ntoa
            (iph[8])
        destination = socket.inet_
            ntoa(iph[9])
        protocol = iph[6]

        print("source IP:", source)
        print("Destination IP:",
                destination)
        print("Protocol:", protocol)
        print("-----------")

except KeyboardInterrupt:
    print("Sniffer stopped.")
finally:
    s.close()
```

Output:

Packet Sniffer Started....
source IP: 127.0.0.1
Destination IP: 127.0.0.1
protocol: 6
- - - - - - - - - - - - -

RESULT:

Thus a packet-sniffing program using raw sockets was implemented to display source IP, destination IP and protocol information.